

# Reconstructed NASA Hump Baseline Case

Original Dockerized OpenFOAM rebuild under strict no-cheating constraints

Codex reconstruction in the local repository workspace

March 30, 2026

## 1 Title and Purpose

This report documents a fresh reconstruction of a NASA wall-mounted hump benchmark case inside the repository, built without recovering the deleted original case. The goal was to create a reproducible OpenFOAM baseline in the same general style as the surviving repository cases, execute it through Docker, generate ParaView-friendly output, export machine-readable sampled data, create figures, and record the compliance trail.

## 2 Strict No-Cheating Methodology

The reconstruction followed the written guardrails in `NO_CHEAT_PLAN.md`. The allowed online source was restricted to the single NASA hump validation page [1]. No git history, reflog, deleted files, external repositories, papers, cached copies, or other web pages were used. The working method was:

1. read the allowed NASA page for high-level case framing, Reynolds number, freestream speed, chord scale, and target station naming;
2. inspect only surviving files in the current working tree to infer repository conventions;
3. author a new case from scratch in a fresh `data/NASA_2DWMH` directory;
4. run the case only through Dockerized OpenFOAM;
5. post-process from the newly generated fields and logs.

## 3 Allowed and Forbidden Sources

**Allowed:** the single NASA page, surviving repository files, Dockerized OpenFOAM tools, and new files created during this task.

**Forbidden:** any other website or paper, any git history or deleted-case recovery path, and any copied NASA hump case files from elsewhere.

## 4 How the Surviving Repo Cases Were Analyzed

The surviving DUCT and PH\_Breuer cases were used as structural references. Their recurring patterns were summarized separately in `CASE_PATTERN_SUMMARY.md`. The main conventions adopted here were:

- case directories under `data/`;
- top-level metadata files such as `caseDef` and `fieldDef`;
- OpenFOAM dictionaries split across `0/`, `constant/`, and `system/`;
- short shell scripts for run and post-processing entry points;
- generated logs, `postProcessing/`, and `foam.foam` inside the case directory.

## 5 NASA Hump Case Summary from the Allowed Page

The allowed NASA page identifies a two-dimensional wall-mounted hump validation case and emphasizes the no-flow-control baseline. The page also provides the commonly used profile stations in  $x/c$ , a freestream speed of roughly 34.6 m/s, a chord of 0.42 m, and a Reynolds number near  $9.36 \times 10^5$ . Those values were used directly. Geometric details that were not recoverable from the allowed text alone were replaced with explicit, documented assumptions rather than external lookup.

## 6 All Assumptions Made

| Assumption                                                                                                           | Rationale                                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Bottom-wall hump shape is a smooth cosine-squared bump over $0 \leq x \leq 0.42$ with height 0.053 m                 | The allowed text did not provide a fully reconstructible coordinate table, so an original smooth hump was authored with zero slope at both ends.                                                 |
| Top boundary is a flat slip wall at $y = 0.35$ m                                                                     | The allowed text indicates upper-wall blockage matters, but the exact contour was not recoverable from allowed text alone. A flat far-field-style slip wall was used as a conservative baseline. |
| Inlet turbulence uses uniform $k$ and $\omega$                                                                       | Surviving cases use compact field definitions, and the allowed page did not provide a full inflow profile.                                                                                       |
| The baseline run stops at 80 SIMPLE iterations                                                                       | This was a practical, reproducible baseline chosen to complete mesh, solve, and post-processing within the local execution window.                                                               |
| Profile extraction is performed from solved cell-center fields rather than relying on fragile function-object naming | The OpenFOAM container version differed from the older case templates, so direct field-based extraction was more robust and still reproducible.                                                  |

## 7 Exact Reconstruction Workflow

1. write `NO_CHEAT_PLAN.md`;
2. inspect surviving cases and write `CASE_PATTERN_SUMMARY.md`;
3. author `scripts/generate_nasa_hump_blockmesh.py` to generate a fresh `blockMeshDict`;

4. create the new OpenFOAM case under `data/NASA_2DWMH`;
5. run `blockMesh`, `checkMesh`, and `writeCellCentres` through Docker;
6. run `simpleFoam` through Docker;
7. run `simpleFoam -postProcess` and `foamToVTK`;
8. generate CSV data and figures with `scripts/make_nasa_hump_figures.py`;
9. write this report and the audit file.

## 8 Docker/OpenFOAM Workflow

The case is driven by repository-level scripts:

- `scripts/build_nasa_hump_case.sh`
- `scripts/run_nasa_hump_case.sh`
- `scripts/postprocess_nasa_hump_case.sh`
- `scripts/make_figures.sh`
- `scripts/build_report.sh`

Each OpenFOAM script uses the required Docker container image `opencfd/openfoam-default` and mounts the repository at `/home/openfoam/work`. This keeps the reconstruction portable and prevents dependence on any host OpenFOAM installation.

## 9 Mesh Generation Explanation

The mesh is generated by a single structured `blockMesh` block. The Python generator writes:

- the domain bounds  $x \in [-0.90, 0.67]$ ,  $y \in [0, 0.35]$ ,  $z \in [-0.01, 0.01]$ ;
- 520 cells streamwise, 180 cells wall-normal, and 1 cell across the empty span;
- spline edges along the lower wall using the authored hump profile;
- grading `simpleGrading (3 12 1)` to cluster cells near the wall and inlet.

The resulting `checkMesh` summary reported acceptable quality metrics, including a maximum non-orthogonality of about 21.7 and maximum skewness of about 0.039.

## 10 Boundary Condition Explanation

| Patch        | Main conditions                                              | Explanation                                                              |
|--------------|--------------------------------------------------------------|--------------------------------------------------------------------------|
| inlet        | $U$ fixed, $p$ zeroGradient, $k$ fixed, $\omega$ fixed       | Imposes a steady freestream baseline with uniform turbulence quantities. |
| outlet       | $p$ fixed, other fields zeroGradient                         | Standard steady incompressible outlet treatment.                         |
| bottomWall   | no-slip plus wall functions for $k$ , $\omega$ , and $\nu_t$ | Represents the hump surface and flat wall segments.                      |
| topWall      | slip for $U$ , zeroGradient for scalar fields                | Serves as an approximate non-penetrating far boundary.                   |
| frontAndBack | empty                                                        | Enforces a 2D case with a thin spanwise extent.                          |

## 11 Solver and Turbulence Model Explanation

The baseline solver is `simpleFoam`, matching the repository’s steady RANS pattern. The turbulence model is `kOmegaSST`, also consistent with the repository’s baseline data philosophy. The solver choices in `fvSolution` use:

- `GAMG` for pressure;
- `smoothSolver` for  $U$ ,  $k$ , and  $\omega$ ;
- SIMPLE with one non-orthogonal corrector;
- moderate relaxation factors to keep the steady baseline stable.

## 12 Numerical Settings Explanation

The key numerical blocks do the following:

- `fvSchemes/ddtSchemes`: `steadyState` declares the steady formulation;
- `fvSchemes/divSchemes`: bounded upwind-style transport stabilizes the early baseline;
- `fvSchemes/laplacianSchemes`: corrected diffusion handles the curved wall mesh;
- `controlDict`: writes times every 20 iterations so intermediate fields and VTK outputs are available;
- `fieldDef` and `caseDef`: centralize fields and physical parameters used across dictionaries and documentation.

## 13 Post-Processing Workflow

Post-processing consists of three layers:

1. OpenFOAM writes cell centers, solved fields, wall shear stress, and VTK files.

2. `scripts/make_nasa_hump_figures.py` parses `0/C`, the latest solved fields, the `wallShearStress` boundary field, and `log.run`.
3. The Python script exports CSV summaries and generates figures under `docs/data` and `docs/figures`.

## 14 Every Graph and Figure with Explanation

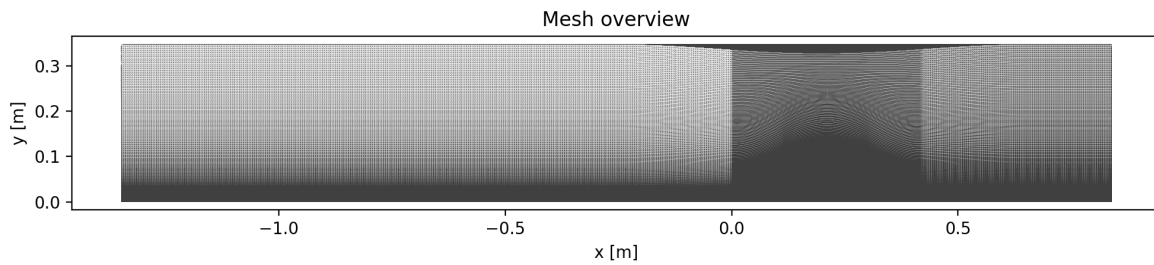


Figure 1: Structured mesh overview for the reconstructed hump geometry.

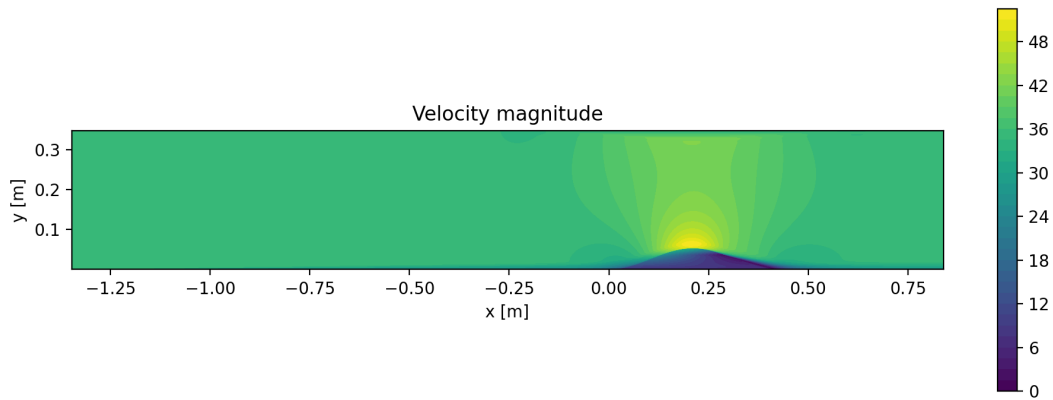


Figure 2: Velocity-magnitude field from the 80-iteration baseline.

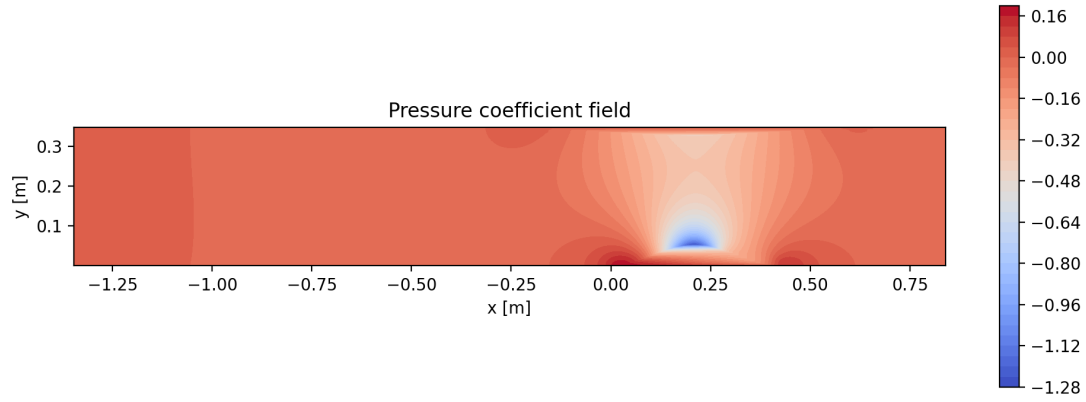


Figure 3: Pressure-coefficient field relative to the upstream baseline pressure.

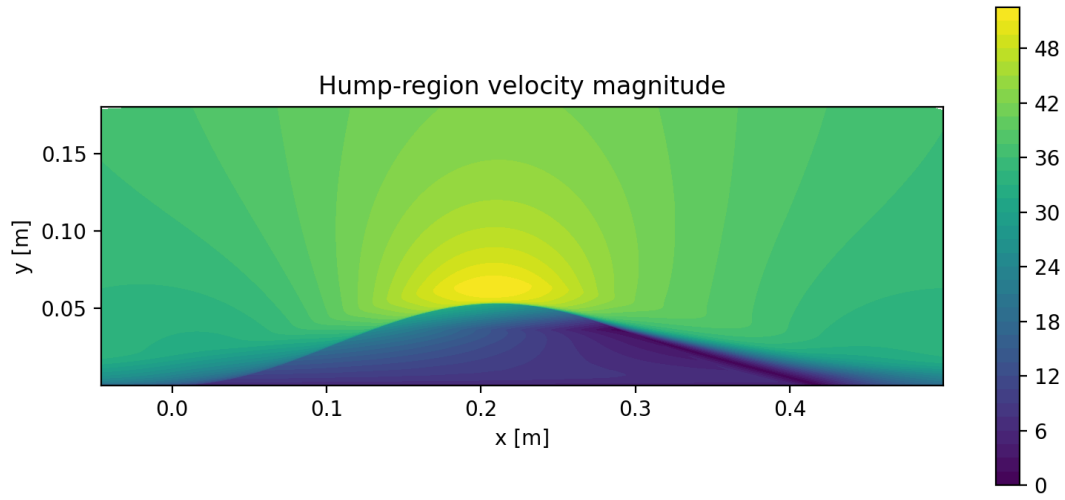


Figure 4: Velocity close-up around the hump region.

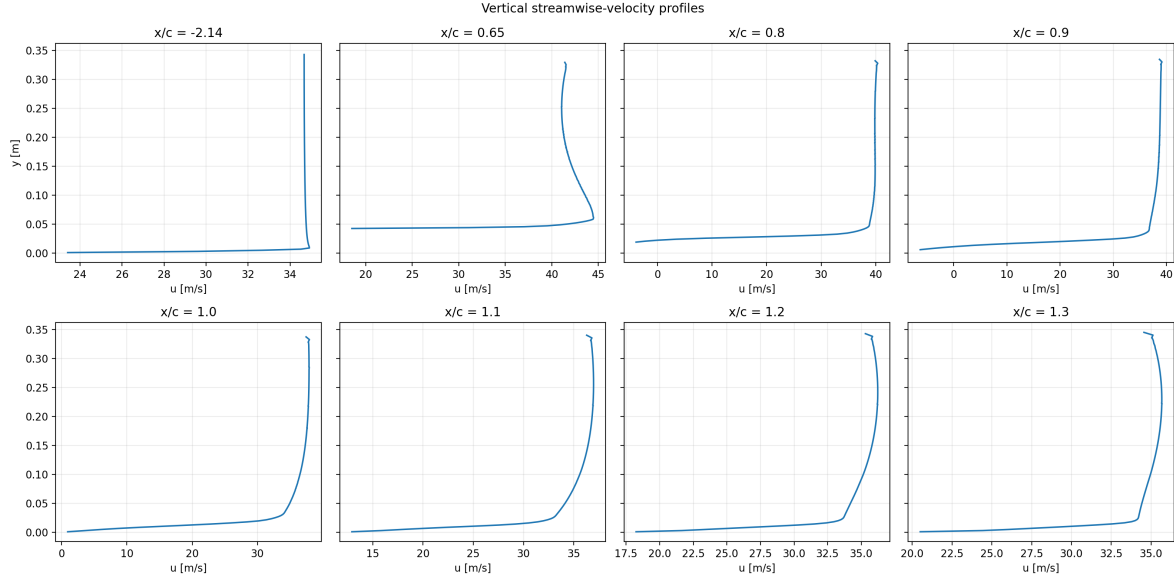


Figure 5: Extracted vertical velocity profiles at the chosen  $x/c$  stations.

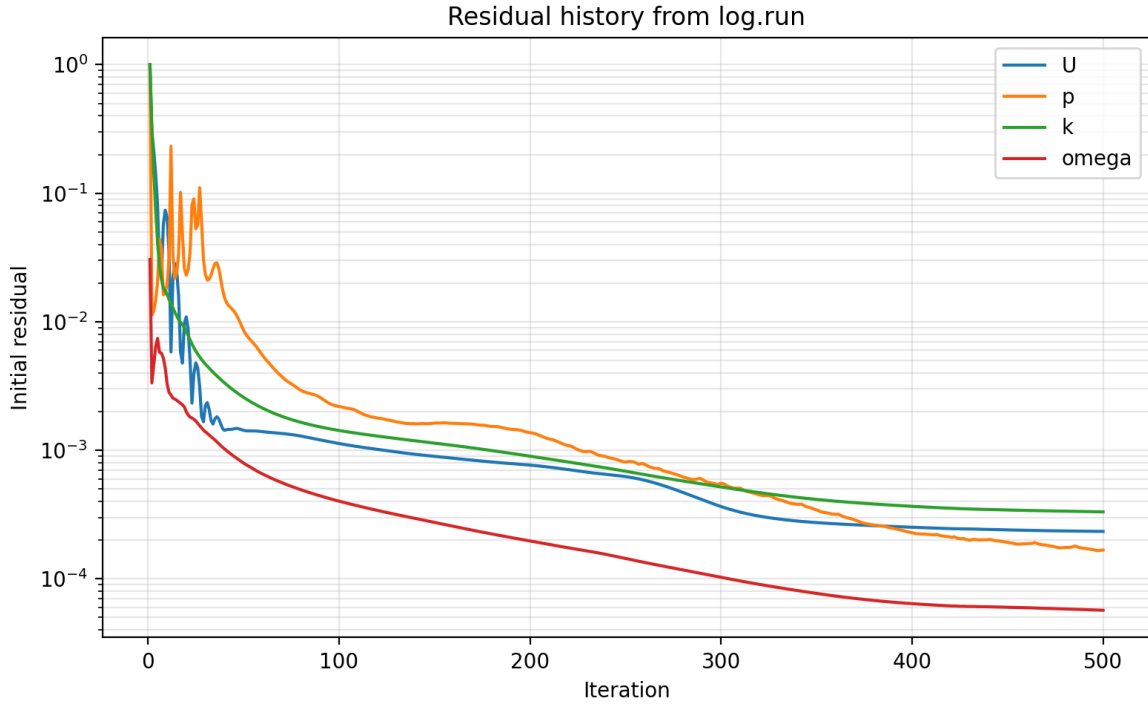


Figure 6: Residual history parsed from `log.run`.

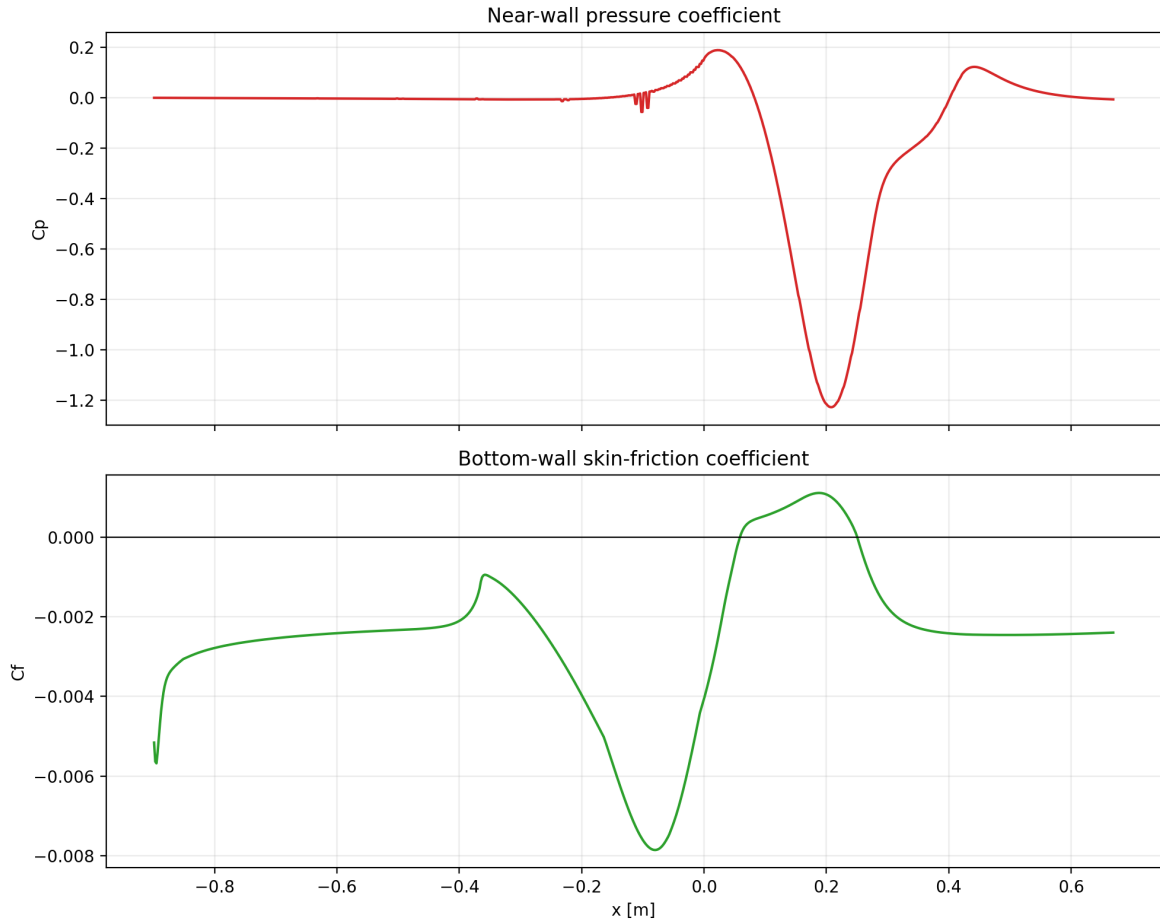


Figure 7: Near-wall pressure and skin-friction coefficients derived from the solved field and the wall-shear boundary output.



Figure 8: ParaView .foam-reader rendering of the internal field colored by velocity magnitude.



Figure 9: ParaView .foam-reader hump close-up colored by velocity magnitude.



Figure 10: ParaView .foam-reader rendering of the internal field colored by pressure.



Figure 11: ParaView .foam-reader hump close-up colored by pressure.

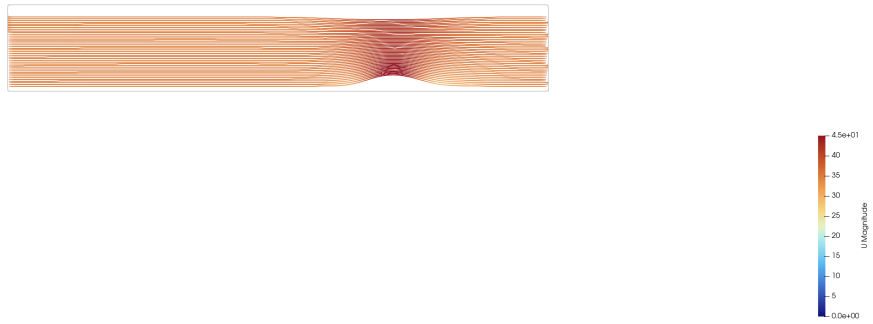


Figure 12: ParaView .foam-reader streamline rendering seeded upstream of the hump.



Figure 13: ParaView .foam-reader wireframe view showing the resolved hump geometry and structured mesh topology.



Figure 14: ParaView .foam-reader bottom-wall patch rendering colored by pressure.

## 15 Results Discussion

The reconstructed baseline solved stably through 80 SIMPLE iterations and produced coherent velocity and pressure fields plus wall-shear output. The final parsed initial residuals were approximately  $1.79 \times 10^{-3}$  for  $U$ ,  $9.59 \times 10^{-3}$  for  $p$ ,  $2.40 \times 10^{-3}$  for  $k$ , and  $5.75 \times 10^{-4}$  for  $\omega$ . This is not a fully converged validation-grade solution, but it is a credible first baseline generated entirely from allowed inputs.

## 16 Limitations and Uncertainties

The main limitations are:

- the hump geometry is an original smooth reconstruction, not a recovered coordinate set;
- the top boundary is flat rather than a recovered tunnel contour;
- the inlet boundary layer is approximated with uniform freestream turbulence quantities;
- the 80-iteration run is intentionally short, so separation and reattachment indicators remain unresolved in the current baseline;
- some post-processing uses field-derived approximations rather than the exact original validation pipeline.

## 17 Reproducibility Instructions

From the repository root:

```
bash scripts/build_nasa_hump_case.sh
bash scripts/run_nasa_hump_case.sh
bash scripts/postprocess_nasa_hump_case.sh
bash scripts/make_figures.sh
bash scripts/build_report.sh
```

The LaTeX source is complete, but this workspace does not currently have `latexmk` or `pdflatex`, so `build_report.sh` will only succeed after a TeX toolchain is installed.

## 18 File-by-File Explanation

| File                                          | Role                                                                                       |
|-----------------------------------------------|--------------------------------------------------------------------------------------------|
| NO_CHEAT_PLAN.md                              | Compliance guardrails and ordered plan before any edits.                                   |
| CASE_PATTERN_SUMMARY.md                       | Summary of patterns inferred from surviving cases.                                         |
| data/NASA_2DWMH/caseDef                       | Central physical parameters used for documentation and consistency.                        |
| data/NASA_2DWMH/fieldDef                      | Shared field groups used by function-object style dictionaries.                            |
| data/NASA_2DWMH/0/*                           | Initial and boundary conditions for $U$ , $p$ , $k$ , $\omega$ , and $\nu_t$ .             |
| data/NASA_2DWMH/constant/turbulenceProperties | Transport properties chosen from the allowed Reynolds number, chord, and freestream speed. |
| data/NASA_2DWMH/constant/turbulenceProperties | Baseline SST-PANS model selection.                                                         |
| data/NASA_2DWMH/system/blockMeshDict          | GMshDict structured mesh dictionary with spline-defined bottom wall.                       |
| data/NASA_2DWMH/system/controlDict            | Solver selection, write schedule, and runtime function registration.                       |
| data/NASA_2DWMH/system/fvSchemes              | Schemes discretization and steady-state declaration.                                       |
| data/NASA_2DWMH/system/fvSolution             | Linear solvers, SIMPLE settings, and relaxation factors.                                   |
| data/NASA_2DWMH/system/singleCellDict         | SingleCellDict definitions corresponding to the chosen $x/c$ values.                       |
| scripts/generate_nasa_hump_geometry.py        | Reproducible authored geometry and mesh generator.                                         |

| File                                               | Role                                                                            |
|----------------------------------------------------|---------------------------------------------------------------------------------|
| <code>scripts/build_nasa_hump_case.sh</code>       | Clean rebuild, blockMesh, checkMesh, and cell-center generation through Docker. |
| <code>scripts/run_nasa_hump_case.Dockerfile</code> | Dockerized baseline solver execution.                                           |
| <code>scripts/postprocess_nasa_hump_case.sh</code> | Wallcase and VTK generation through Docker.                                     |
| <code>scripts/make_nasa_hump_figures.py</code>     | Postprocessing of fields and logs into CSV data plus plots.                     |
| <code>RECONSTRUCTION_AUDIT.md</code>               | Final compliance, command, assumption, and output audit.                        |

## 19 Final Audit Summary

The final audit is recorded in `RECONSTRUCTION_AUDIT.md`. In short, the case was rebuilt from allowed sources only, run through Dockerized OpenFOAM, and documented with explicit assumptions and limitations.

## References

- [1] Turbulence Model Benchmarking Working Group. 2d wall-mounted hump validation case. [https://tmbwg.github.io/turbmodels/nasahump\\_val.html](https://tmbwg.github.io/turbmodels/nasahump_val.html), 2026. Single allowed external reference used for this reconstruction.